

RELATÓRIO DE AUDITORIA E BENCHMARKING DE ALGORITMOS DE ORDENAÇÃO

**Diego Pereira Nadur Castro
Davi da Silva Lorena
João Pedro Garcia Torezan
Theo Fellipe Silva Santos**

Relatório Técnico de Disciplina do Curso
Bacharelado em Ciência da Computação



Castro, Diego Pereira Nadur

Relatório de Auditoria e Benchmarking de Algoritmos de Ordenação / Diego Pereira Nadur Castro; Davi da Silva Lorena; João Pedro Garcia Torezan; Theo Fellipe Silva Santos. - Itajubá: UNIFEI, 2026.

11 p.

Relatório Técnico de Disciplina do Curso (Ciência da Computação) - Universidade Federal de Itajubá, Itajubá, 2026

Orientação: Vanessa Souza

1. Benchmarking. 2. Algoritmos de Ordenação. 3. Auditoria de Software. 4. Complexidade Assintótica. I. Souza, Vanessa, orient. II. Título.

FOLHA DE APROVAÇÃO

Castro, Diego Pereira Nadur; Lorena, Davi da Silva; Torezan, João Pedro Garcia; Santos, Theo Felipe Silva. **Relatório de Auditoria e Benchmarking de Algoritmos de Ordenação**. 11 p. Relatório Técnico de Disciplina do Curso (Ciência da Computação), Universidade Federal de Itajubá, Itajubá, 2026.

Defesa em 2026.

BANCA EXAMINADORA:

Prof. Vanessa Souza



Relatório de Auditoria e Benchmarking de Algoritmos de Ordenação

Diego Pereira Nadur Castro, Davi da Silva Lorena, João Pedro Garcia Torezan, Theo
Fellipe Silva Santos

Orientadora: Vanessa Souza

RESUMO

Resumo Este relatório apresenta os processos e resultados da auditoria técnica realizada na biblioteca de ordenação `libordenacao7.a`, desenvolvida por terceiros. Motivado por uma solicitação de uma Consultoria de Qualidade de Software (QA) que identificou possíveis anomalias nas implementações, o trabalho foi conduzido por estudantes da Universidade Federal de Itajubá (UNIFEI) no âmbito da disciplina de Algoritmos e Estruturas de Dados II. A metodologia baseia-se em benchmarking técnico com métricas de tempo de execução, comparações e movimentações de registros, além de verificação de estabilidade dos algoritmos. Foram analisados os algoritmos Bolha Inteligente, Seleção, Inserção, MergeSort, QuickSort Tradicional, Shell-Sort, HeapSort e BozoSort. Os resultados revelaram anomalias em algoritmos específicos, com contradições entre as métricas reportadas e o comportamento real observado. Como resultado, o grupo desenvolveu implementações corrigidas e um laudo de auditoria detalhado com hipóteses sobre as causas raiz das falhas identificadas.

Palavras-chave: Benchmarking; Algoritmos de Ordenação; Auditoria de Software; Complexidade Assintótica

1 Introdução

Este relatório apresenta o processo e os resultados da auditoria técnica realizada na biblioteca de ordenação `libordenacao7.a`. O projeto foi motivado por uma solicitação de uma Consultoria de Qualidade de Software (QA), que identificou possíveis anomalias nas implementações desenvolvidas por terceiros. A suspeita principal recai sobre inconsistências de desempenho e contagem de operações que violariam as expectativas teóricas de complexidade assintótica e estabilidade dos algoritmos.

Para conduzir esta investigação experimental, a consultoria estabeleceu uma parceria estratégica com estudantes da **Universidade Federal de Itajubá (UNIFEI)**, especificamente no âmbito da disciplina de Algoritmos e Estruturas de Dados II. A equipe de acadêmicos assume o papel de auditores de QA, com a missão de realizar testes rigorosos e diagnosticar falhas sistêmicas através de evidências numéricas.

A metodologia adotada para a execução deste projeto baseia-se nos seguintes pilares:

- **Abordagem Investigativa:** O foco não reside na implementação inicial, mas na execução de experimentos e análise crítica dos resultados fornecidos pela biblioteca.
- **Benchmarking Técnico:** Desenvolvimento de um sistema robusto em linguagem C capaz de extrair métricas reais de tempo de execução, comparações de chaves e movimentações de registros.
- **Aprendizagem Baseada em Problemas (ABP):** Utilização de uma metodologia ativa que incentiva a colaboração em equipe, a formulação de hipóteses e a tomada de

decisões informadas para a resolução de problemas complexos.

O objetivo final deste trabalho é fornecer um laudo de auditoria preciso, identificando anomalias e propondo versões corretivas que garantam a eficiência e a confiabilidade exigidas pela consultoria de software.

2 Metodologia

2.1 Especificação de Hardware e Software

Para execução dos testes e análise dos resultados obtidos, foi utilizada uma máquina com a seguinte especificação:

- **Equipamento:** Laptop Samsung Galaxy Book 4
- **Processador:** Intel Core i5
- **Memória RAM:** 8 GB
- **Armazenamento:** 512 GB
- **Sistema Operacional:** Windows 11 Home/Pro (64 bits)

2.2 Confirmação dos Testes

Após implementação da biblioteca `libordenacao7.a`, por meio de uma biblioteca estática disponibilizada pela empresa — onde o código desenvolvido pelos antigos desenvolvedores foi incorporado ao executável na compilação —, os alunos desenvolveram um programa para testes dos diferentes casos de ordenação, entre os quais estavam a ordenação aleatória e quase ordenada.

Para estes dois cenários em específico, os membros acordaram que seriam necessárias análises para uma média de 30 repetições com conjuntos diferentes, a fim de obter melhores resultados e, consequentemente, compreensão de possíveis problemas existentes nas implementações. Para isso, foram desenvolvidas funções para gerar vetores relacionadas às respectivas circunstâncias, os quais se diferenciavam devido à utilização de uma “semente” dinâmica (`time(NULL)`), que se modificava a cada iteração do código. Tal conteúdo foi aprendido pelos universitários em seu curso, relacionado à aleatoriedade de conjuntos na perspectiva computacional, com aplicações diretas em Inteligências Artificiais, Jogos, Simulações e Criptografia em sistemas robustos de segurança.

2.3 Verificação de Estabilidade

A verificação de estabilidade baseia-se na distinção fundamental entre o valor da chave e a cronologia do registro. A metodologia utiliza a estrutura `request`, onde o campo `user_id` funciona como a chave de ordenação primária e o campo `chegada` serve como um identificador único de posição original.

O processo inicia-se com a geração de um vetor de teste com colisões propositalmente (chaves duplicadas) e a aplicação do algoritmo implementado, que desordena o vetor inicial para garantir que a auditoria não seja influenciada por uma ordenação prévia favorável. Essa etapa é crucial para extrair métricas reais de movimentações e evitar anomalias de desempenho que poderiam mascarar instabilidades.

Após a execução do algoritmo auditado, o sistema realiza uma varredura por pares para verificar se a ordem relativa foi mantida. Tecnicamente, isso significa que, para quaisquer dois elementos que possuam o mesmo `user_id`, aquele com o menor valor de `chegada` deve obrigatoriamente aparecer antes no vetor ordenado. Se o algoritmo desrespeita essa sequência, ele é classificado como instável, o que é um fator crítico para garantir a confiabilidade exigida pela consultoria de software em sistemas que dependem da integridade histórica dos dados.

```
r* gerar_aleatorio(int tam) {
    return geraAleatorios(tam, time(NULL));
}
```

Figura 1: Função para gerar aleatório

```
r* gerar_quase_ordenado(int tam) {
    return geraQuaseOrdenados(tam, time(NULL), 10);
}
```

Figura 2: Função para gerar quase ordenado

3 Resultados

A seguir estão registradas as tabelas e gráficos correspondentes aos testes dos algoritmos de ordenação disponibilizados na biblioteca `libordenacao7.a`. Cada algoritmo possui 3 tabelas relacionadas às métricas de tempo, médias de comparações e movimentações.

3.1 Tabelas:

3.1.1 Bolha Inteligente

Tabela 1: Bolha Inteligente – Tempo em (ms).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	1,33	0,00	1,00	1,13
2	10.000	110,80	0,00	126,00	84,33
3	50.000	5.678,00	0,00	3.131,00	1.482,87
4	75.000	11.253,50	0,00	7.084,00	2.499,93
5	100.000	18.497,53	0,00	12.862,00	4.051,73

Tabela 2: Bolha Inteligente – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	498,372	999	499,500	499,122
2	10.000	49.985,462	9.999	49.995,000	49.932,714
3	50.000	1.249.931,677	49.999	1.249.974,997	1.094.320,848
4	75.000	2.812.391,272	74.999	2.812.462,500	1.904.826,384
5	100.000	4.998.983,984	99.999	4.999.499,997	2.718.262,917

Tabela 3: Bolha Inteligente – Média de Movimentações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	508,880	0	998,038	112,400
2	10.000	50.128,732	0	99.979,804	11.657,340
3	50.000	1.249.876,136	0	2.499.873,836	176.032,764
4	75.000	2.810.605,249	0	5.624.752,878	238.517,559
5	100.000	4.998.028,011	0	9.999.593,998	29.014,225

3.1.2 Seleção

Tabela 4: Seleção – Tempo em (ms).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	0,63	0,00	1,00	0,50
2	10.000	66,63	54,00	77,00	54,13
3	50.000	1.194,77	1.206,00	3.079,00	1.192,97
4	75.000	3.459,50	2.823,00	6.499,00	2.761,47
5	100.000	4.620,87	4.628,00	10.201,00	4.786,53

Tabela 5: Seleção – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	499,499	499,499	499,499	499,499
2	10.000	49.994,999	49.994,999	49.994,999	49.994,999
3	50.000	1.249.974,999	1.249.974,999	1.249.974,999	1.249.974,999
4	75.000	2.812.462,499	2.812.462,499	2.812.462,499	2.812.462,499
5	100.000	4.999.949,999	4.999.949,999	4.999.949,999	4.999.949,999

Tabela 6: Seleção – Média de Movimentações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	1,988	0	1,222	276
2	10.000	19,981	0	12,470	2,992
3	50.000	99,976	0	63,528	16,739
4	75.000	149,974	0	98,444	29,088
5	100.000	199,974	0	134,064	41,527

3.1.3 Inserção

Tabela 7: Inserção – Tempo em (ms).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	0,57	0,00	0,00	0,13
2	10.000	59,80	0,00	120,00	13,83
3	50.000	1.298,87	0,00	2.600,00	183,03
4	75.000	3.029,43	0,00	5.998,00	252,13
5	100.000	5.799,93	0,00	12.130,00	310,00

Tabela 8: Inserção – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	260,783	999	499,386	60,481
2	10.000	24.957,317	9.999	49.993,581	5.662,147
3	50.000	625.333,913	49.999	1.249.961,272	878.225,248
4	75.000	1.406.108,019	74.999	2.812.422,179	5.624.828,199
5	100.000	2.501.769,688	99.999	4.999.865,666	145.132,262

Tabela 9: Inserção – Média de Movimentações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	520.578	0	999.082	119.929
2	10.000	49.904.643	0	99.989.655	11.314.283
3	50.000	1.250.617.833	0	2.499.923.979	175.583.251
4	75.000	2.812.630.928	0	5.624.828.121	238.611.053
5	100.000	500.349.384	0	9.999.693.691	290.097.281

Tabela 17: ShellSort – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	15.249	8.006	12.499	12.955
2	10.000	268.390	120.005	180.599	220.720
3	50.000	1.903.125	700.006	1.076.496	1.286.653
4	75.000	2.841.012	1.125.008	1.667.849	1.637.484
5	100.000	4.299.296	1.500.006	2.273.503	2.133.493

3.1.4 MergeSort

Tabela 10: MergeSort – Tempo em (ms).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	0,10	0,00	1,00	0,17
2	10.000	1,40	1,00	1,00	1,00
3	50.000	7,40	4,00	4,00	4,87
4	75.000	12,63	7,00	7,00	8,17
5	100.000	16,23	9,00	10,00	10,67

Tabela 18: ShellSort – Média de Movimentações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	11.167	0	8.752	7.785
2	10.000	205.571	0	120.672	146.747
3	50.000	1.556.346	0	744.337	778.241
4	75.000	22.714.455	0	1.077.760	706.430
5	100.000	3.561.273	0	1.536.191	835.896

3.1.7 HeapSort

Tabela 19: HeapSort – Tempo em (ms).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	0,10	0,00	0,00	0,03
2	10.000	1,37	1,00	1,00	1,10
3	50.000	7,03	8,00	6,00	5,80
4	75.000	11,57	8,00	8,00	7,90
5	100.000	16,20	14,00	12,00	13,07

Tabela 11: MergeSort – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	8.697	5.044	5.146	8.073
2	10.000	120.413	69.008	67.290	115.080
3	50.000	718.055	401.952	402.026	581.885
4	75.000	1.120.580	624.316	630.460	811.428
5	100.000	1.536.665	853.904	878.172	1.040.446

Tabela 12: MergeSort – Média de Movimentações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	19.952	19.952	19.952	19.952
2	10.000	267.232	267.232	267.232	267.232
3	50.000	1.568.928	1.568.928	1.568.928	1.568.928
4	75.000	2.437.856	2.437.856	2.437.856	2.437.856
5	100.000	3.337.856	3.337.856	3.337.856	3.337.856

Tabela 20: HeapSort – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	16.847	17.544	15.957	17.366
2	10.000	235.396	244.418	226.612	242.524
3	50.000	1.409.609	1.456.787	1.365.976	1.449.812
4	75.000	2.200.427	2.270.298	2.138.583	2.261.500
5	100.000	3.019.237	3.115.298	296.481	3.104.065

Tabela 21: HeapSort – Média de Movimentações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	18.082	19.320	16.592	19.100
2	10.000	248.406	262.392	2.333.278	259.954
3	50.000	1.474.556	1.548.144	1.397.912	1.539.006
4	75.000	2.297.894	2.406.740	2.190.188	2.396.478
5	100.000	3.148.837	3.296.726	2.995.478	3.284.292

3.1.5 QuickSort

Tabela 13: QuickSort Tradicional – Tempo em (ms).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	0,07	0,00	1,00	0,03
2	10.000	0,83	51,00	63,00	0,63
3	15.000	1,10	120,00	130,00	1,27
4	25.000	2,00	291,00	339,00	2,00
5	30.000	2,40	405,00	485,00	2,30

Tabela 14: QuickSort Tradicional – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	10.675	499.500	403.942	145.692
2	10.000	161.489	49.995.000	39.522.000	289.596
3	15.000	258.351	112.492.500	89.408.222	665.842
4	25.000	418.207	312.487.500	256.672.347	1.203.839
5	30.000	514.106	449.985.000	356.585.949	1.146.676

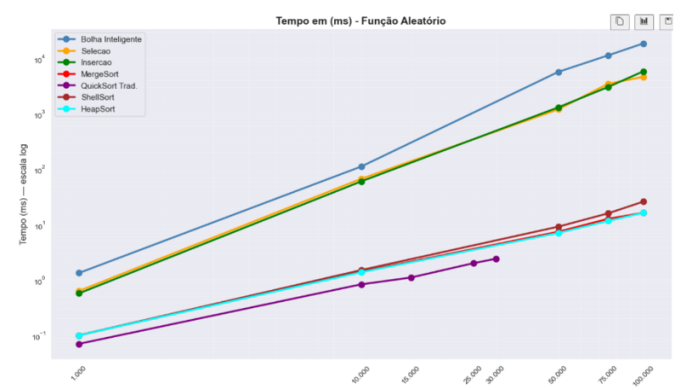
Tabela 15: QuickSort Tradicional – Média de Movimentações.

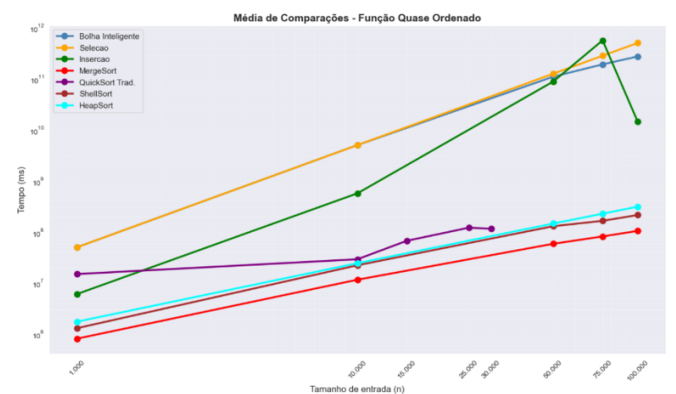
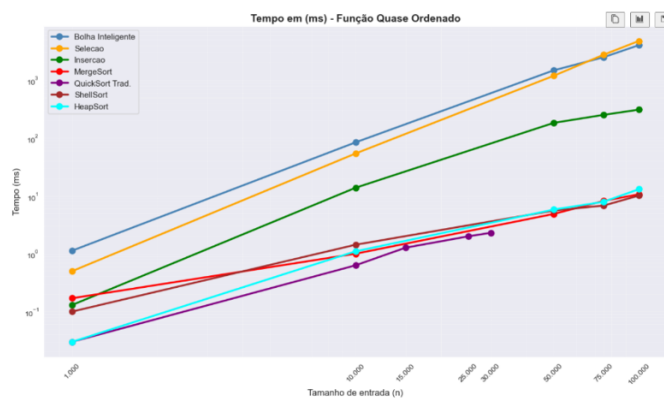
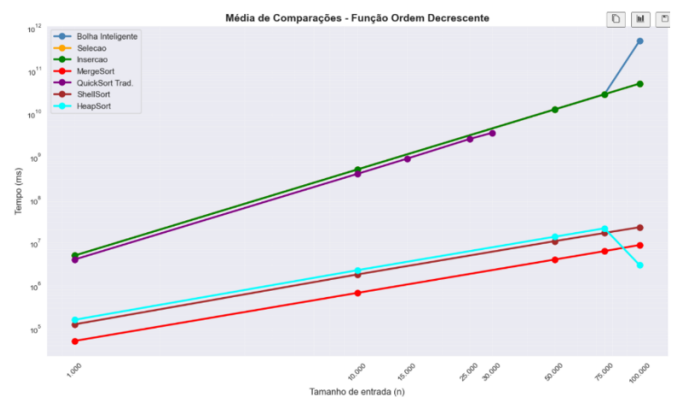
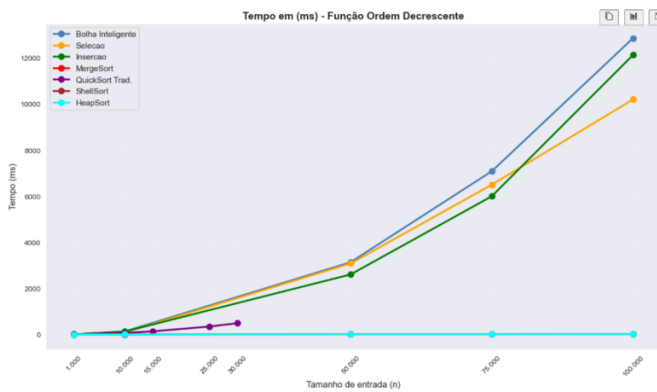
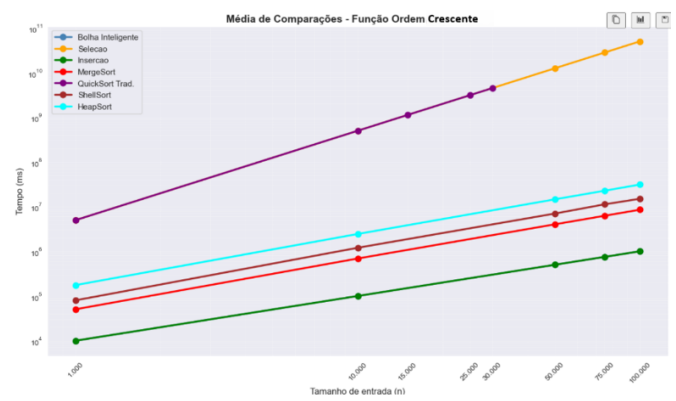
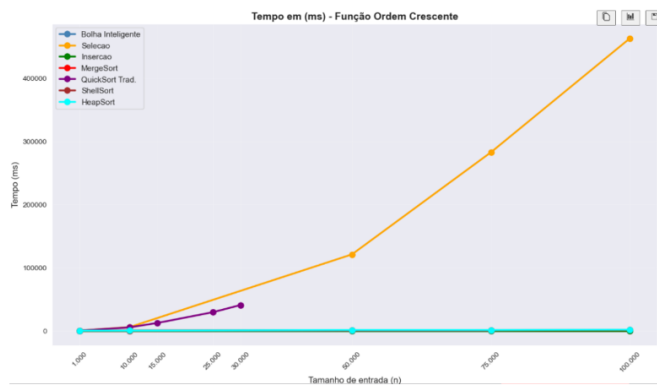
Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	9.892	1.998	148.692	5.964
2	10.000	154.534	19.998	15.544.594	104.184
3	15.000	211.484	29.998	34.392.142	171.002
4	25.000	412.280	49.998	85.309.116	285.474
5	30.000	470.964	59.998	138.205.790	369.404

3.1.6 ShellSort

Tabela 16: ShellSort – Tempo em (ms).

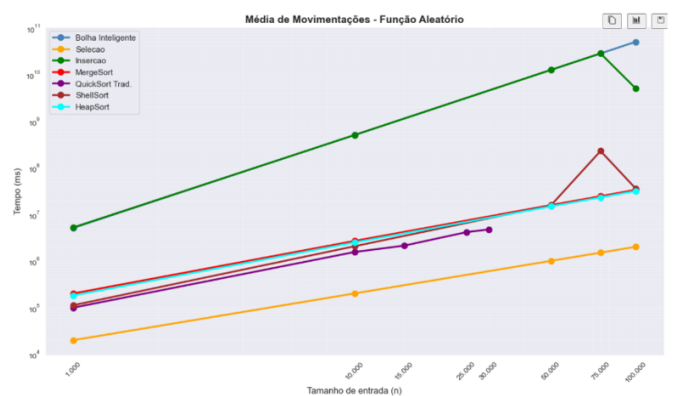
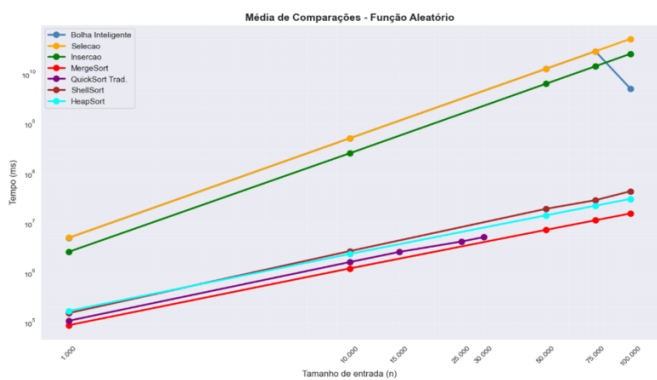
Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	0,10	0,00	0,00	0,10
2	10.000	1,50	0,00	0,00	1,43
3	50.000	9,13	2,00	3,00	5,57
4	75.000	15,87	3,00	3,00	6,80
5	100.000	25,83	9,00	9,00	10,13





3.2.2 Média de Comparações:

3.2.3 Média de Movimentações:



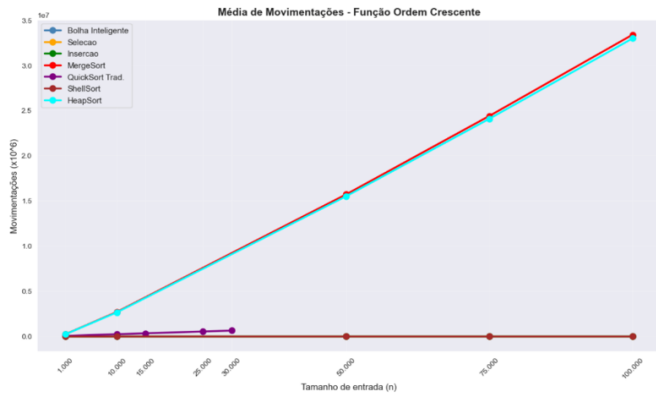


Tabela 23: MergeSort (vetores grandes) – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	8.697	5.044	5.146	8.073
2	10.000	120.413	69.008	67.290	115.080
3	100.000	1.536.665	853.904	878.172	1.040.446
4	1.000.000	18.674.331	10.066.432	10.911.276	10.264.475
5	10.000.000	2.200.099.645	118.788.160	128.534.814	118.979.168

Tabela 24: MergeSort (vetores grandes) – Média de Movimentações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	19.952	19.952	19.952	19.952
2	10.000	267.232	267.232	267.232	267.232
3	100.000	3.337.856	3.337.856	3.337.856	3.337.856
4	1.000.000	39.902.848	39.902.848	39.902.848	39.902.848
5	10.000.000	466.444.556	466.445.568	466.445.568	466.445.568

3.3.2 ShellSort – Vetores Grandes

Tabela 25: ShellSort (vetores grandes) – Tempo em (ms).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	0,10	0,00	0,00	0,10
2	10.000	1,50	0,00	0,00	1,43
3	100.000	25,83	9,00	9,00	10,13
4	1.000.000	318,63	53,00	79,00	47,67
5	10.000.000	4.346,37	518,00	813,00	799,93

Tabela 26: ShellSort (vetores grandes) – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	15.249	8.006	12.499	12.955
2	10.000	268.390	120.005	180.599	220.720
3	100.000	4.299.296	1.500.006	2.273.503	2.133.493
4	1.000.000	64.884.250	18.000.007	25.898.302	18.563.224
5	10.000.000	960.852.902	220.000.008	299.762.568	220.343.721

Tabela 27: ShellSort (vetores grandes) – Média de Movimentações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	11.167	0	8.752	7.785
2	10.000	205.571	0	120.672	146.747
3	100.000	3.561.273	0	1.536.191	835.896
4	1.000.000	55.256.454	0	15.768.596	743.338
5	10.000.000	840.255.008	0	159.494.869	481.219

3.3.3 HeapSort – Vetores Grandes

Tabela 28: HeapSort (vetores grandes) – Tempo em (ms).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	0,10	0,00	0,00	0,03
2	10.000	1,37	1,00	1,00	1,10
3	100.000	16,20	14,00	12,00	13,07
4	1.000.000	247,03	145,00	154,00	145,83
5	10.000.000	4.400,07	1.522,00	1.723,00	1.537,50

Tabela 29: HeapSort (vetores grandes) – Média de Comparações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	16.847	17.544	15.957	17.366
2	10.000	235.396	244.418	226.612	242.524
3	100.000	3.019.237	3.115.298	296.481	3.104.065
4	1.000.000	36.794.098	36.002.897	35.948.096	36.009.270
5	10.000.000	434.637.217	398.463.304	425.592.683	398.463.304

Tabela 30: HeapSort (vetores grandes) – Média de Movimentações.

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	18.082	19.320	16.592	19.100
2	10.000	248.406	262.392	2.333.278	259.954
3	100.000	3.148.837	3.296.726	2.995.478	3.284.292
4	1.000.000	38.095.906	37.599.852	36.599.534	37.619.208
5	10.000.000	447.659.900	411.528.806	432.159.098	411.591.959

3.3 Tabelas para Vetores Maiores:

Abaixo estão representadas tabelas relacionadas aos testes com vetores maiores, os quais puderam ser realizados nos algoritmos MergeSort, ShellSort e HeapSort.

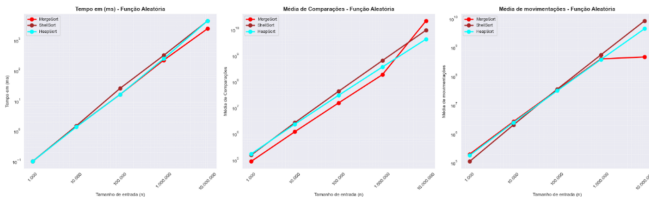
3.3.1 MergeSort – Vetores Grandes

Tabela 22: MergeSort (vetores grandes) – Tempo em (ms).

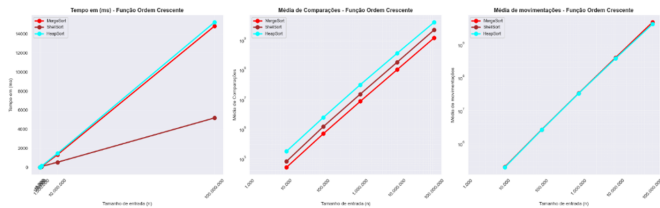
Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	0,10	0,00	1,00	0,17
2	10.000	1,40	1,00	1,00	1,00
3	100.000	16,23	9,00	10,00	10,67
4	1.000.000	221,27	132,00	137,00	137,27
5	10.000.000	2.416,50	1.482,00	1.473,00	1.528,40

3.4 Gráficos para vetores Maiores

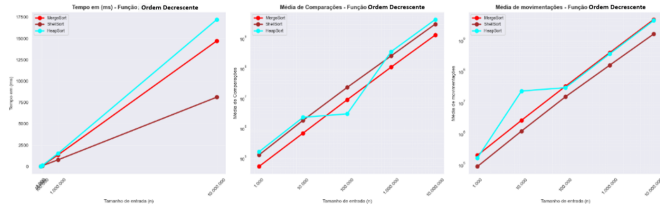
3.4.1 Função Aleatória



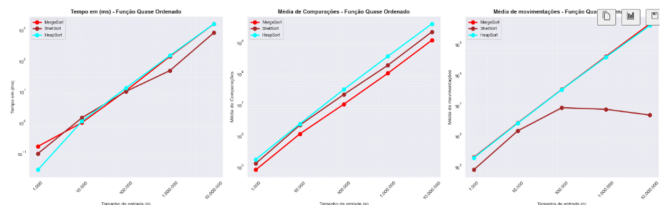
3.4.2 Função Crescente



3.4.3 Função Decrescente



3.4.4 Função Quase Ordenada



4 Laudo de Auditoria

4.1 Caso 1: Algoritmo de Seleção

O algoritmo Selection Sort possui uma característica matemática imutável em sua complexidade teórica: o número de comparações realizadas independe do estado inicial de ordenação do vetor. Para qualquer cenário (aleatório, ordenado, crescente ou quase ordenado), a rotina varre o conjunto não ordenado repetidas vezes, devendo seguir estritamente a equação assintótica de comparações $C(n) = \frac{n(n-1)}{2}$.

Tabela 31: Seleção – Média de Comparações (reprodução da Tabela 5).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado (média 30 rep.)
1	1.000	499.499	499.499	499.499	499.499
2	10.000	49.994.999	49.994.999	49.994.999	49.994.999
3	50.000	1.249.974.999	1.249.974.999	1.249.974.999	1.249.974.999
4	75.000	2.812.462.499	2.812.462.499	2.812.462.499	2.812.462.499
5	100.000	4.999.949.999	4.999.949.999	4.999.949.999	4.999.949.999

- **Evidência Numérica:** A Tabela 31 atesta que, para um vetor de tamanho $n = 1.000$, o sistema da biblioteca registrou exatamente 499.499 comparações em todos os cenários. Contudo, aplicando a fórmula teórica para $n = 1.000$, o valor exato deveria ser $\frac{1000 \times 999}{2} = 499.500$. A ausência sistemática de exatamente uma (1) comparação em todos os testes expõe uma falha estrutural no código implementado.

- **Hipótese da Causa Raiz:** A auditoria aponta para a existência de um erro crítico na lógica na condição de parada do laço externo, com a suposição de que tenha sido implementado como `for (int i = 0; i < n - 2; i++)`. Esta instrução aborta a execução uma iteração antes do término correto, omitindo a comparação final entre o penúltimo e o último registro.

Exemplo Prático (vetor de N = 20 elementos):

Tabela 32: Comparativo do estado do vetor ($N = 20$, semente = 67) após ordenação por Seleção.

Vetor Original	Seleção (Ineficiente)	Seleção (Corrigida)
17 - 0	1 - 2	1 - 2
6 - 1	1 - 13	1 - 13
1 - 2	3 - 4	3 - 4
7 - 3	4 - 5	4 - 5
3 - 4	4 - 14	4 - 14
4 - 5	6 - 7	6 - 7
11 - 6	6 - 1	6 - 1
6 - 7	7 - 3	7 - 3
15 - 8	7 - 9	7 - 9
7 - 9	7 - 12	7 - 12
11 - 10	9 - 19	9 - 19
12 - 11	10 - 16	10 - 16
7 - 12	11 - 6	11 - 6
1 - 13	11 - 17	11 - 17
4 - 14	11 - 18	11 - 18
13 - 15	11 - 10	11 - 10
10 - 16	12 - 11	12 - 11
11 - 17	13 - 15	13 - 15
11 - 18	17 - 0	15 - 8
9 - 19	15 - 8	17 - 0

*Nota: A formatação dos dados apresenta o padrão **Valor - Chegada**, em que

o primeiro número representa a chave de ordenação (*user_id*) e o segundo indica a ordem cronológica original do registro (*chegada*).

Tabela 33: Comparativo de métricas para o vetor de $N = 20$.

Métrica	Seleção (Ineficiente)	Seleção (Corrigida)
Comparações	189	190
Movimentações	32	51
Tempo (ms)	0.00	0.00

4.2 Caso 2: Algoritmo de Inserção

O algoritmo de Inserção possui uma assimetria matemática em sua complexidade teórica que depende do estado inicial de ordenação do vetor. Diferente do Seleção, a rotina apresenta sua eficiência máxima ($O(n)$) no melhor cenário (vetor já ordenado) e sofre uma severa degradação quadrática ($O(n^2)$) no

pior cenário (vetor em ordem decrescente), varrendo exaustivamente os elementos à esquerda.

Tabela 34: Inserção – Média de Comparações (reprodução da Tabela 8).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado
1	1.000	260.783	999	499.386	60.481
2	10.000	24.957.317	9.999	49.993.581	5.662.147

- **Evidência Numérica:** A tabela atesta o abismo de complexidade do algoritmo: nos cenários de Ordem Crescente, o sistema registrou exatas 999 e 9.999 comparações (a varredura ideal de $n - 1$), ratificando o comportamento linear perfeito[cite: 2]. Em contrapartida, no cenário de Ordem Decrescente, as comparações saltaram para 499.386 (Teste 1) e 49.993.581 (Teste 2). No entanto, aplicando a fórmula matemática universal do pior caso $C(n) = \frac{n(n-1)}{2}$, os resultados exatos e obrigatórios seriam 499.500 e 49.995.000, respectivamente. A ausência sistemática de comparações expõe uma anomalia estrutural no código da biblioteca.

- **Hipótese da Causa Raiz:** A auditoria aponta para a existência de um erro crítico na instrumentação das métricas no código-fonte oculto do método:

Contagem Inconsistente e Deslocamento Cego: A ausência das comparações exatas ocorre, provavelmente, porque o incrementador de métricas (`m->comparacoes++`) foi posicionado de forma equivocada dentro do laço interno (`while`). Isso faz com que o algoritmo encerre o ciclo antes de o incrementador conseguir registrar todas as comparações que o processador efetivamente avaliou, resultando em dados métricos que não batem com a realidade matemática da Ciência da Computação.

4.3 Caso 3: Algoritmo QuickSort Tradicional

O algoritmo QuickSort é amplamente reconhecido por sua alta eficiência média ($O(n \log n)$). Contudo, sua estabilidade estrutural depende inteiramente da estratégia de particionamento (escolha do pivô). Em implementações ingênuas, vetores já perfeitamente ordenados (crescente ou decrescente) forçam o algoritmo ao seu pior caso teórico, degradando a performance para $O(n^2)$ e sobrecarregando perigosamente a memória de recursão do sistema.

Tabela 35: QuickSort Tradicional – Média de Comparações (reprodução da Tabela 14).

Teste	Tamanho	Aleatório (média 30 rep.)	Ordem Crescente	Ordem Decrescente	Quase Ordenado
1	1.000	10.675	499.500	403.942	148.692
2	10.000	161.489	49.995.000	39.522.000	289.596
3	15.000	258.351	112.492.500	89.408.222	665.842
4	25.000	418.207	312.487.500	256.672.347	1.203.839
5	30.000	514.106	449.985.000	356.585.949	1.146.676

- **Evidência Numérica:** Diferente de algoritmos como MergeSort e HeapSort, que foram validados com vetores de até 100.000 (e posteriormente 10.000.000) posições, a bateria de testes do QuickSort precisou ser abortada precocemente no tamanho 30.000. Analisando a métrica de comparações para 30.000 elementos em Ordem

Crescente, o registro atinge absurdos 449.985.000. Esse valor espelha perfeitamente a matemática de um comportamento quadrático ($O(n^2)$). Quando submetido a uma entrada superior, como 100.000, a versão ineficiente da biblioteca sequer finaliza a execução, resultando em travamento total do programa, enquanto a versão corrigida conclui a ordenação sem anomalias.

- **Hipótese da Causa Raiz:** A auditoria diagnóstica que o travamento da aplicação não é um simples engasgo de processamento, mas sim uma vulnerabilidade arquitetônica fatal conhecida como pior caso recursivo extremo:

Pivô Ingênuo e Estouro de Pilha: A biblioteca ineficiente implementa a escolha do pivô de maneira estática e ingênuo (fixando sempre o primeiro ou o último elemento da partição). Quando submetido a um vetor que já está em ordem Crescente ou Decrescente, essa escolha estática gera partições totalmente desbalanceadas em todas as iterações. Isso ultrapassa drasticamente o limite da pilha de chamadas do sistema operacional, causando o travamento imediato (*Stack Overflow*).

5 Correção do Algoritmo

5.1 Trecho do Algoritmo Implementado

Em virtude das falhas detectadas em alguns algoritmos de ordenação elaborados anteriormente por terceiros, os acadêmicos da UNIFEI decidiram desenvolver algoritmos corretos com todas as implementações referentes aos casos de distribuição usados pela empresa. Para isso, foi implementado um Tipo Abstrato de Dados (TAD), com o intuito de encapsular dados e operações, separando a especificação da implementação. Isso organiza o código, facilita a manutenção, reutilização e oculta detalhes complexos de estruturação, permitindo que o usuário interaja apenas com uma interface definida, além de assegurar confiança, restringindo o acesso direto aos registros importantes.

As aplicações já haviam sido estudadas pelos alunos na disciplina de **Algoritmos e Estrutura de Dados II**. Sendo assim, era necessário apenas moldar os códigos de maneira que se aplicassem ao que é utilizado pela consultoria, tais como as Notas e IDs, respeitada a preferência de ordenação.

5.1.1 Estrutura de Dados e Métricas

A implementação utiliza uma estrutura de dados denominada *request*, que simula uma requisição com um identificador de usuário (`user_id`) e um marcador de ordem original (*chegada*). Além disso, foi implementada uma estrutura *metrics* para contabilizar de forma exata o número de comparações e movimentações, permitindo uma análise quantitativa da eficiência.

```
//STRUCTS
struct metricas{
    unsigned Long Long comparacoes;
    unsigned Long Long movimentacoes;
};

struct request {
    int user_id;
    int chegada;
};
```

5.1.2 Descrição da Implementação: Selection Sort (Seleção)

O algoritmo Selection Sort foi implementado para exemplificar o comportamento de métodos de ordenação por seleção com complexidade quadrática ($O(n^2)$). A lógica fundamental consiste em dividir o vetor em duas partes: uma sublista de itens já ordenados e outra de itens a ordenar.

A implementação utiliza dois laços de repetição alinhados. O laço externo percorre o vetor posição por posição, enquanto o laço interno busca o índice do menor elemento (menor) no restante do vetor. Uma vez identificado o menor valor da sequência não ordenada, o algoritmo realiza uma troca direta com o elemento da posição atual do laço externo.

Como Selection Sort é inerentemente instável, as trocas ocorrem entre elementos distantes. Assim, o algoritmo pode "saltar" sobre elementos com chaves iguais (`user_id`), alterando sua ordem relativa de chegada.

O código original de referência apresentava uma condição de parada prematura no laço externo (`for (int i = 0; i < n - 2; i++)`), o que interrompia a varredura uma iteração antes do término correto. Esse erro lógico resultava em exatamente uma comparação a menos do que o exigido matematicamente e deixava os últimos elementos do vetor suscetíveis à desordenação. A implementação atual corrigiu esse limite de iteração, garantindo que o penúltimo elemento seja devidamente comparado com o último, assegurando a ordenação completa do conjunto.

```
met *selecao(r *vet, int tam) {
    met *m = alocaMetricas();
    int menor;
    r aux;
    for (int i = 0; i < tam; i++) {
        menor = i;
        for (int j = i + 1; j < tam; j++) {
            m->comparacoes++;
            if (vet[j].user_id < vet[menor].user_id) {
                menor = j;
            }
        }
        if (menor != i){
            aux = vet[i];
            vet[i] = vet[menor];
            vet[menor] = aux;
            m->movimentacoes+=3;
        }
    }
    return m;
}
```

5.2 Prova de Estabilidade

A estabilidade em algoritmos de ordenação é a propriedade que garante que elementos com chaves idênticas mantenham sua ordem relativa original após o processamento. No sistema implementado, isso é verificado através do campo `chegada`. Se dois elementos possuem o mesmo `user_id`, aquele com o menor valor de chegada deve aparecer primeiro no vetor ordenado.

5.2.1 Análise dos Resultados Experimentais

Os logs obtidos através dos testes de estresse comprovam que o comportamento dos algoritmos está em total conformidade com suas propriedades teóricas, o que valida a correção da implementação.

```
===== VERIFICACAO DE ESTABILIDADE =====

[Estabilidade] Bolha Inteligente    (tam= 100): ESTAVEL
[Estabilidade] Selecao              (tam= 100): NAO ESTAVEL (108 violacoes)
[Estabilidade] Insercao             (tam= 100): ESTAVEL
[Estabilidade] MergeSort            (tam= 100): ESTAVEL
[Estabilidade] QuickSort            (tam= 100): NAO ESTAVEL (198 violacoes)
[Estabilidade] ShellSort            (tam= 100): NAO ESTAVEL (217 violacoes)
[Estabilidade] HeapSort             (tam= 100): NAO ESTAVEL (234 violacoes)
```

5.2.2 Conclusão da Verificação

A análise dos logs permite concluir que não existem erros de implementação no que tange à estabilidade. As “violações” reportadas nos algoritmos **Seleção**, **QuickSort**, **ShellSort** e **HeapSort** não são falhas de código, mas sim consequências de suas lógicas intrínsecas:

- **Algoritmos Estáveis (Bolha, Inserção, Merge):** Confirmaram 0 violações, pois realizam trocas apenas entre elementos adjacentes ou possuem critérios de desempate controlados durante a intercalação.
- **Algoritmos Instáveis (Seleção, Quick, Shell, Heap):** As violações comprovam que estes métodos realizam trocas de longa distância ou partições que “saltam” elementos iguais, perdendo a ordem original de chegada.

Portanto, a implementação é considerada robusta e fidedigna, uma vez que replicou exatamente os comportamentos esperados na literatura da Ciência da Computação, confirmando que o sistema de métricas é capaz de distinguir com precisão a natureza de cada método de ordenação.

6 Análise do Caso BozoSort

Abaixo estão representadas tabelas relacionadas às métricas de tempo, médias de comparações e movimentações para dois diferentes testes/casos analisados após execução do algoritmo BozoSort, implementado na biblioteca `libordenacao7.a`, com o intuito de validar a aleatoriedade e diferenças registradas para vetores de mesmo tamanho, as quais serão explicadas melhor posteriormente.

Tabela 36: BozoSort – Tempo em (ms).

Teste	Tamanho	Aleatório (média 30 rep.)		Ordem Crescente		Ordem Decrescente		Quase Ordenado (média 30 rep.)	
		Caso 1	Caso 2	Caso 1	Caso 2	Caso 1	Caso 2	Caso 1	Caso 2
1	4	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,00
2	8	0,40	0,20	0,00	0,00	0,00	0,40	0,00	0,00
3	10	18,20	21,80	0,00	0,00	0,80	0,40	23,60	0,00
4	12	124,80	241,40	0,00	0,00	34,40	55,80	0,00	139,60

Tabela 37: BozoSort – Média de Comparações.

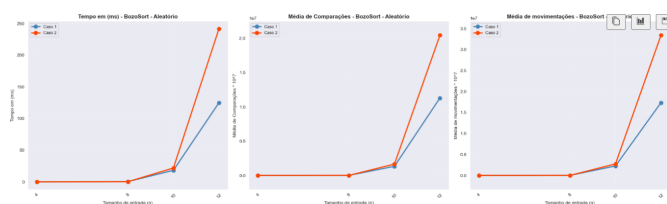
Teste	Tamanho	Aleatório (média 30 rep.)		Ordem Crescente		Ordem Decrescente		Quase Ordenado (média 30 rep.)	
		Caso 1	Caso 2	Caso 1	Caso 2	Caso 1	Caso 2	Caso 1	Caso 2
1	4	25	39	3	3	51	28	3	3
2	8	15.217	9.068	7	7	6.223	12.485	7	19.545
3	10	1.336.192	1.661.129	9	9	72.895	37.592	1.827.358	9
4	12	11.296.439	20.434.105	11	11	3.039.780	5.054.122	11	12.858.244

Tabela 38: BozoSort – Média de Movimentações.

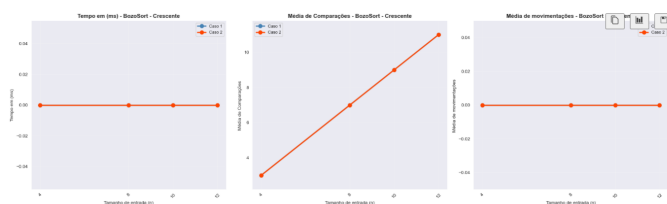
Teste	Tamanho	Aleatório (média 30 rep.)		Ordem Crescente		Ordem Decrescente		Quase Ordenado (média 30 rep.)	
		Caso 1	Caso 2	Caso 1	Caso 2	Caso 1	Caso 2	Caso 1	Caso 2
1	4	42	66	0	0	75	42	0	0
2	8	25.176	14.019	0	0	10.242	19.545	0	0
3	10	2.264.025	2.750.907	0	0	123.330	62.952	3.074.802	0
4	12	17.337.738	33.392.527	0	0	4.665.123	8.096.181	0	20.536.288

Em seguida, seguem os gráficos das métricas para cada tipo de ordenação:

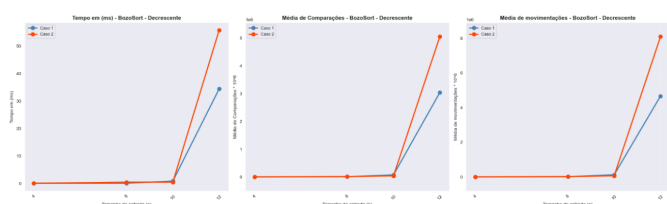
• Aleatória



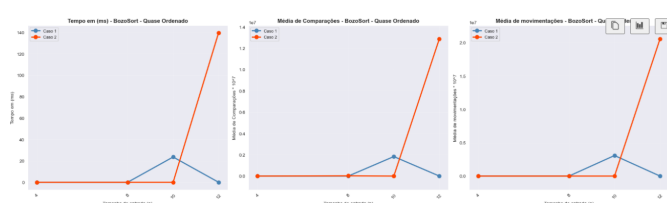
• Crescente



• Decrescente



• Quase Ordenada



Maior tamanho de vetor ordenado em até 5 minutos: 14

De acordo com vários testes efetuados, chegou-se à conclusão de que seria possível obter as métricas e verificar o comportamento do sistema até o vetor de tamanho 14, sendo este

o máximo que o computador do grupo conseguiu ordenar no tempo limite de cinco minutos. Após isso, foi identificada uma “explosão combinatória”, visto que levaria um tempo considerável para a obtenção dos dados referentes ao vetor de tamanho 15, ultrapassando muito além do limite esperado.

6.1 Viabilidade do Algoritmo

Consoante as verificações analisadas pelo grupo, conclui-se que o Algoritmo BozoSort **não é viável** para a empresa, tendo em vista determinados fatores que restringem ao máximo sua eficiência em um ambiente de produção:

- **Vasta Complexidade:** O BozoSort tem uma complexidade média de aproximadamente $O(n \cdot n!)$, ou seja, para um vetor de tamanho n , existem $n!$ permutações possíveis, de modo que a probabilidade de encontrar a única permutação correta através de trocas aleatórias cresce de forma fatorial. Ademais, visto que o algoritmo é probabilístico, não há garantia teórica de que ele terminará em um tempo finito. Ele pode, teoricamente, continuar trocando elementos e nunca atingir o estado ordenado, o que resulta em uma complexidade ilimitada.
- **Aleatoriedade do Tempo de Execução:** Como o algoritmo escolhe dois índices aleatórios e os troca, não existe uma estratégia definida. Desse modo, ele pode dar “sorte” em um vetor maior de elementos e encontrar a ordenação mais rápida do que em um vetor menor, ou então registrar tempos extremamente distintos para um mesmo tamanho de vetor, sem contar o número de comparações e movimentações. Seria como uma aposta na loteria: um usuário pode ganhar com apenas 1 bilhete, outro pode jogar 1000 e não ganhar nada.

7 Considerações Finais

7.1 Conclusões Técnicas

A auditoria e o *benchmarking* realizados comprovaram a importância vital da validação algorítmica antes da integração de bibliotecas de terceiros em ambientes de produção. O estudo demonstrou empiricamente as propriedades assintóticas dos métodos:

- O **MergeSort** destacou-se como o mais consistente, mantendo complexidade $O(n \log n)$ e garantindo estabilidade em todos os cenários testados, tornando-se a escolha ideal para manipulação de grandes volumes de requisições.
- O **QuickSort** (versão tradicional) apresentou o melhor tempo de execução absoluto para vetores aleatórios, mas evidenciou vulnerabilidade a pior caso ($O(n^2)$) em distribuições ordenadas/decrescentes e quebra de estabilidade.
- Algoritmos quadráticos ($O(n^2)$) como **Bolha**, **Inserção e Seleção** mostraram-se viáveis apenas para entradas ínfimas, com o Seleção sendo o único a não apresentar otimização temporal para vetores já ordenados.

- A análise do **BozoSort** evidenciou a inviabilidade comercial de algoritmos com comportamento estocástico fatorial ($O(n * n!)$), reprovando seu uso em qualquer escopo de software real.

Por fim, a detecção de falhas lógicas e de corrupção de dados em implementações supostamente padronizadas (Seleção, Inserção e ShellSort originais) ratificou que métricas reportadas isoladamente não garantem integridade estrutural, exigindo testes rigorosos de caixa-preta e caixa-branca.

7.2 Reflexão sobre a Metodologia

A utilização da Aprendizagem Baseada em Problemas (ABP) foi determinante para o sucesso deste projeto. A transição do papel de "alunos executores" para "auditores investigadores de Software (QA)" estimulou o pensamento crítico e a busca autônoma por embasamento teórico. O desafio de analisar uma biblioteca estática (caixa-preta) exigiu o desenvolvimento criativo de testes de estresse e rastreabilidade, extrapolando a mera codificação. A divisão de tarefas entre os membros da equipe funcionou de maneira coesa, permitindo a elaboração paralela do sistema de testes, a depuração matemática das complexidades e a formulação deste laudo técnico formal. A experiência uniu teoria acadêmica e realidade de forma altamente satisfatória.

7.3 Link do Repositório

O código implementado e comentado está disponível no repositório do grupo: <https://github.com/Davi-Lorena/trabalho-aedii-grupo7>

Referências

